

Overview

This document is a compact Markdown companion to the full Doxygen HTML output. It lists the documented classes, structs, files and their public API members.

Contents

- [class ApplModule](#applmodule)
- [class AsyncMqttClient](#asyncmqttclient)
- [class AsyncWebSocket](#asynwebsocket)
- [class AsyncWebSocketClient](#asynwebsocketclient)
- [class AsyncWebSocketMessageBuffer](#asynwebsocketmessagebuffer)
- [class CAccessToken](#caccesstoken)
- [class CAESCryptor](#caescryptor)
- [class CAESToken](#caestoken)
- [class CAppl](#cappl)
- [class CBase64Data](#cbase64data)
- [class CBatteryMeasure](#cbatterymeasure)
- [class CButton](#cbutton)
- [class CConfigHandler](#cconfighandler)
- [class CEventHandler](#ceventhandler)
- [class CEventLogger](#ceventlogger)
- [class CFS](#cfs)
- [class CInputPinController](#cinputpincontroller)
- [class CJsonNode](#cjsonnode)
- [class CLightSwitch](#clightswitch)
- [class CLogWriter](#clogwriter)
- [class CMDNSController](#cmdnscontroller)
- [class CMQTTController](#cmqttcontroller)
- [class CMultiNameUniqueValueTable](#cmultinameuniquevaluetable)
- [class CNamedValueEntry](#cnamedvalueentry)
- [class CNamedValueTable](#cnamedvaluetable)
- [class CNTPHandler](#cntphandler)
- [class COutputPinController](#coutputpincontroller)
- [class CPinController](#cpincontroller)
- [class CRGBLed](#crgbled)
- [class CSerialLogWriter](#cseriallogwriter)
- [class CSimpleDelay](#csimpledelay)
- [class CStatusHandler](#cstatushandler)
- [class CStreamLogWriter](#cstreamlogwriter)
- [class CSysStatus](#csysstatus)
- [class CSysStatusLed](#csysstatusled)
- [class CUniqueNameMultiValueTable](#cuniquenamemultivaluetable)
- [class CVar](#cvar)
- [class CVarTable](#cvarTable)
- [class CWebServer](#cwebserver)
- [class CWebSocket](#cwebsocket)
- [class CWebSocketMessage](#cwebsocketmessage)

- [class CWiFiController](#cwificontroller)
- [class IConfigHandler](#confighandler)
- [class IHomeAssistantComponent](#homeassistantcomponent)
- [class IModule](#module)
- [class IMessageEventReceiver](#msgeventreceiver)
- [class IPAddress](#ipaddress)
- [class IStatusHandler](#statushandler)
- [class MQTTMessage](#mqttmessage)
- [class NativeSerial](#nativeserial)
- [file AccessToken.h](#accesstoken.h)
- [file AESEncryptor.h](#aesencryptor.h)
- [file Appl.h](#appl.h)
- [file ApplModule.h](#applmodule.h)
- [file base64.h](#base64.h)
- [file Base64Data.h](#base64data.h)
- [file BatteryMeasure.h](#batterymeasure.h)
- [file Button.h](#button.h)
- [file ConfigHandler.h](#confighandler.h)
- [file CWebSocket.md](#websocket.md)
- [file DevelopmentHelper.h](#developmenthelper.h)
- [file EventHandler.h](#eventhandler.h)
- [file FileSystem.h](#filesystem.h)
- [file InputPinController.h](#inputpincontroller.h)
- [file JsonHelper.h](#jsonhelper.h)
- [file JsonNode.h](#jsonnode.h)
- [file LightSwitch.h](#lightswitch.h)
- [file Logging.h](#logging.h)
- [file LSCUtils.h](#lscutils.h)
- [file mainpage.md](#mainpage.md)
- [file MDNSController.h](#mdnscontroller.h)
- [file ModuleInterface.h](#moduleinterface.h)
- [file MQTTController.h](#mqttcontroller.h)
- [file Msgs.h](#msgs.h)
- [file NamedValueTable.h](#namedvaluetable.h)
- [file Network.h](#network.h)
- [file NTPHandler.h](#ntp handler.h)
- [file OutputPinController.h](#outputpincontroller.h)
- [file pageBuilder.md](#pagebuilder.md)
- [file PinController.h](#pincontroller.h)
- [file ProgVersion.h](#progversion.h)
- [file README.md](#readme.md)
- [file RGBLed.h](#rgbled.h)
- [file Runtime.h](#runtime.h)
- [file Security.h](#security.h)
- [file SimpleDelay.h](#simpledelay.h)
- [file StatusHandler.h](#statushandler.h)
- [file SysStatus.h](#sysstatus.h)
- [file SysStatusLed.h](#sysstatusled.h)
- [file Vars.h](#vars.h)
- [file WebServer.h](#webserver.h)
- [file WebSocket.h](#websocket.h)
- [file WiFiController.h](#wificontroller.h)
- [namespace LSC](#lsc)
- [namespace LSC_WIFI](#lsc_wifi)

- [struct ApplConfig](#applconfig)
- [struct AsyncMqttClientMessageProperties](#asyncmqttclientmessageproperties)
- [struct AwsFrameInfo](#awsframeinfo)
- [struct CConfigHandler::HandlerEntry](#cconfighandlerhandlerentry)
- [struct CStatusHandler::StatusHandlerEntry](#cstatushandlerstatushandlerentry)
- [struct LSC_FS](#lsc_fs)
- [struct MQTTConfig](#mqttconfig)
- [struct MQTTStatus](#mqttstatus)
- [struct NTPConfig](#ntpconfig)
- [struct WebServerConfig](#webserverconfig)
- [struct WebServerStatus](#webserverstatus)
- [struct WebSocketStatus](#websocketstatus)
- [struct WiFiConfig](#wificonfig)
- [struct WiFiStatus](#wifistatus)

class ApplModule

Abstract Basic Module Implementation You have to implement the dispatch method to reflect your module logic Implement the configuration and status handling.

Scope - Member - Description

public-func - virtual void ApplModule::dispatch() - Default dispatch function by doing nothing.
 public-func - virtual int ApplModule::receiveEvent(const void *pSender, int nMsg, const void *pMessage, int nClass) override - Default Event Receiver, is calling the dispatch function on Application loop.

class AsyncMqttClient

Scope - Member - Description

public-func - bool AsyncMqttClient::connected() -
 public-func - void AsyncMqttClient::connect() -

class AsyncWebSocket

Scope - Member - Description

public-func - AsyncWebSocket::AsyncWebSocket(const char *pszUrl="") -
 public-func - const char * AsyncWebSocket::url() -
 public-func - void AsyncWebSocket::onEvent(...) -
 public-func - void AsyncWebSocket::cleanupClients() -
 public-func - AsyncWebSocketMessageBuffer * AsyncWebSocket::makeBuffer(size_t nSize) -
 public-func - void AsyncWebSocket::textAll(AsyncWebSocketMessageBuffer *pBuffer) -

class AsyncWebSocketClient

Scope - Member - Description

public-attrb - void* AsyncWebSocketClient::_tempObject -

public-func - uint32_t AsyncWebSocketClient::id() -
public-func - IPAddress AsyncWebSocketClient::remoteIP() -
public-func - void AsyncWebSocketClient::text(AsyncWebSocketMessageBuffer *pBuffer) -

class AsyncWebSocketMessageBuffer

Scope - Member - Description

public-func - AsyncWebSocketMessageBuffer::AsyncWebSocketMessageBuffer(size_t nSize) -
public-func - AsyncWebSocketMessageBuffer::~AsyncWebSocketMessageBuffer() -
public-func - uint8_t * AsyncWebSocketMessageBuffer::get() -
public-func - size_t AsyncWebSocketMessageBuffer::size() -

class CAccessToken

Access Token to be exchanged with clients, to ensure correct authentication.

Todoenhance with application specific meta information

Scope - Member - Description

protected-func - bool CAccessToken::loadBase64DataElement(const char *pszData) - Decode and load the base64-encoded token payload.
protected-func - const char * CAccessToken::getBase64DataElement(bool bRefreshTimeStamp=true) - Return the base64-encoded payload, optionally refreshing timestamp.
public-func - CAccessToken::CAccessToken() - Create an empty invalid access token.
public-func - CAccessToken::CAccessToken(const char *pszIPAddress, const char *pszTokenKey, const char *pszIV=nullptr) - Create a token for the given IP address and token key.
public-func - CAccessToken::CAccessToken(const char *pszBase64EncodedString) - Create a token by decoding an existing base64 token string.
public-func - const char * CAccessToken::getTokenAsBase64(bool bRandomIV=true) - Return the complete access token encoded as base64.
public-func - bool CAccessToken::isAuthValid(const char *pszIPAddress, const char *pszTokenKey) - Validate token structure, IP address, key and lifetime.

class CAESCryptor

Helper class for AES encryption/decryption with PKCS#7 padding.

Scope - Member - Description

public-attrb - CAESToken CAESCryptor::IV - Initialization vector used by encrypt/decrypt.
public-attrb - CAESToken CAESCryptor::Passphrase - Default passphrase used when no explicit passphrase is supplied.
public-func - CAESCryptor::CAESCryptor() - Create a cryptor with default IV/passphrase tokens.
public-func - size_t CAESCryptor::decrypt(void *pData, size_t nDataLen, const char *pszPassphrase=nullptr) - Decrypt data in place and return the resulting data length.
public-func - size_t CAESCryptor::encrypt(void *pData, size_t nDataLen, const char *pszPassphrase=nullptr) - Encrypt data in place and return the padded/encrypted length.
public-func - size_t CAESCryptor::insertPkcs7Padding(unsigned char *pData, size_t nDataLength, size_t nBlockSize=AES_CRYPTOR_BLOCK_SIZE) - Add PKCS#7 padding to a buffer and return the padded

length.

public-func - void CAESCryptor::removePkcs7Padding(unsigned char *pData, size_t &nDataLength, size_t nBlockSize=AES_CRYPTOR_BLOCK_SIZE) - Remove PKCS#7 padding and update the data length in place.

class CAESToken

AES Token for IV and Passphrase, used by CAESCryptor.

Scope - Member - Description

public-attrb - unsigned char CAESToken::Data[AES_CRYPTOR_BLOCK_SIZE][AES_CRYPTOR_BLOCK_SIZE] - Raw 16-byte token data used as AES IV/passphrase material.

public-func - CAESToken::CAESToken() - Create a token initialized with empty/default data.

public-func - CAESToken::CAESToken(const char *pszTokenString) - Create a token from a plain string.

public-func - void CAESToken::loadFromString(const char *pszTokenString) - Load token data from a plain string, truncated/padded to block size.

public-func - void CAESToken::loadFromBase64(const char *pszBase64String) - Load token data from a base64-encoded string.

public-func - const char * CAESToken::getAsBase64() - Return token data encoded as base64.

public-func - const char * CAESToken::getAsString() - Return token data as a zero-terminated string buffer.

public-func - void * CAESToken::copyOf() - Return a stable copy of the raw token data.

public-func - void CAESToken::createRandom() - Fill the token with random bytes.

class CAppl

Central application hub for modules, configuration, status and events.

CAppl owns the message bus, central logger, variable table and reusable status documents. Modules register here to participate in config loading/saving, status reporting and application lifecycle events.

Scope - Member - Description

public-attrb - String CAppl::AppName - Human-readable application/firmware name.

public-attrb - String CAppl::AppVersion - Human-readable application/firmware version.

public-attrb - const unsigned long CAppl::StartTime - millis() value captured when the global CAppl object is created.

public-attrb - CVarTable CAppl::Config - Application-local variable table, exported as "cfg".

public-attrb - CEventHandler CAppl::MsgBus - Application message bus used by modules and log writers.

public-attrb - CEventLogger CAppl::Log - Event-based application logger.

public-func - void CAppl::registerModule(const char *pszModuleName, IModule *pModule) - Register a module with configuration, status and event handling.

public-func - void CAppl::writeConfigTo(JsonNode &oNode, bool bHideCritical) override - Write application and registered-module configuration.

public-func - void CAppl::readConfigFrom(JsonNode &oNode) override - Read application and registered-module settings from JSON.

public-func - bool CAppl::readConfigFrom(const char *pszFileName, int nJsonDocSize=JSON_CONFIG_DOC_DEFAULT_SIZE) - Read a configuration file and load settings into all modules.

public-func - void CAppl::migrateConfig(JsonNode &oDoc) - Migrate legacy configuration keys into

their current locations.

public-func - bool CAppl::saveConfig(const char *pszFileName=JSON_APPL_CONFIG_FILE, int nJsonDocSize=JSON_CONFIG_DOC_DEFAULT_SIZE) - Persist the current configuration to the file system.

public-func - JsonNode * CAppl::getStatus(int nLevel=STATUS_LEVEL_INFO) - Build and return the reusable application status node.

public-func - const char * CAppl::getStatusAsText(int nLevel=STATUS_LEVEL_INFO) - Return the current application status serialized as JSON text.

public-func - JsonNode * CAppl::getState() - Return the compact state view of this device.

public-func - const char * CAppl::getStateAsText() - Return the compact device state serialized as JSON text.

public-func - void CAppl::writeStatusTo(JsonNode &oNode, int nStatusLevel) override - Write application metadata and module status into a JSON node.

public-func - void CAppl::writeSystemStatusTo(JsonNode &oNode) - Write lower-level system diagnostics into a JSON node.

public-func - time_t CAppl::getNativeTime() - Return the native system time.

public-func - const char * CAppl::getUpTime() - Return application uptime as HH:MM:SS.

public-func - const char * CAppl::getISODateTime() - Return current local date/time as YYYY-MM-DDTHH:MM:SS.

public-func - const char * CAppl::getISODate() - Return current local date as YYYY-MM-DD.

public-func - const char * CAppl::getISOTime() - Return current local time as HH:MM:SS.

public-func - const char * CAppl::getDeviceID() - Return the stable chip/device identifier from system status.

public-func - String CAppl::getDeviceName() - Return the configured device name.

public-func - void CAppl::setDeviceName(const char *pszName) - Set the configured device name from a C string.

public-func - void CAppl::setDeviceName(String strName) - Set the configured device name from a String.

public-func - void CAppl::setDevicePwd(const char *pszName) - Set the configured device password from a C string.

public-func - void CAppl::setDevicePwd(String strName) - Set the configured device password from a String.

public-func - String CAppl::getDevicePwd() - Return the configured device password.

public-func - CAppl::CAppl() - Create the global application hub.

public-func - void CAppl::dispatch(int nMsgType=0, const void *pMsg=nullptr) - Dispatch a periodic loop message to all registered receivers.

public-func - int CAppl::receiveEvent(const void *pSender, int nMsgType, const void *pMessage, int nClass) - Handle application-level events such as reboot requests.

public-func - void CAppl::init(const char *strAppName, const char *strAppVersion) - Initialize the application metadata and announce startup.

public-func - void CAppl::start(void *pData=nullptr, int nType=0) - Publish the application-started event after initialization finished.

public-func - void CAppl::sayHello() - Print a short application banner to Serial.

public-func - void CAppl::printDiag() - Print flash/heap diagnostics and verify flash size configuration.

public-func - void CAppl::reboot(int nDelayMillis=2000, bool bForce=false) - Send shutdown events and restart the ESP after the given delay.

class CBase64Data

Using the base implementation of NibbleAndHalf (see base64.h).

wrapping the base implementation to be able to change, as in the past, a lot of implementations had problems.

Scope - Member - Description

protected-attr - char* CBase64Data::m_pszEncodedData -
protected-attr - unsigned char* CBase64Data::m_pDecodedData -
public-func - virtual CBase64Data::~CBase64Data() -
public-func - virtual void CBase64Data::clear() -
public-func - const char * CBase64Data::getBase64EncodedString(const char *pszInput, size_t nInputLen=0) -
public-func - const unsigned char * CBase64Data::getBase64DecodedData(const char *pszInput, int &nFinalLen) -
public-static-func - static int CBase64Data::base64EncodeData(const char *pszInput, size_t nInputLen, char *pszOutput, size_t nOutputLen, bool bRemoveCRLF=true) - encode data (also bin) to an base 64 string If output area is not large enough to hold the string, nothing will be changed (!)
public-static-func - static int CBase64Data::base64DecodeData(const char *pszInput, size_t nInputLen, char *pszOutput, size_t nOutputLen) - decode base 64 data to normal representation If output area is not large enough to hold the data, nothing will be changed (!)

class CBatteryMeasure

Scope - Member - Description

public-func - CBatteryMeasure::CBatteryMeasure(int nPin, float fCalcFactor=4.2) - Battery measure constructor.
public-func - bool CBatteryMeasure::isBatteryAvailable() -
public-func - int CBatteryMeasure::getRawMeasureData() -
public-func - float CBatteryMeasure::getVoltage(int nDigits=-1) -
public-func - void CBatteryMeasure::writeStatusTo(JsonNode &oStatusObj, int nLevel) override -
public-func - void CBatteryMeasure::writeConfigTo(JsonNode &oConfigObj, bool bHideCritical) override -
public-func - void CBatteryMeasure::readConfigFrom(JsonNode &oConfigObj) override -

class CButton

Scope - Member - Description

private-func - void IRAM_ATTR CButton::interruptHandler() -
public-func - CButton::CButton() -
public-func - CButton::CButton(int nPin, bool bLowLevellsOn=true, bool bUsePullUpDown=true) -
public-func - CButton::~CButton() -
public-func - void CButton::setup(int nPin, bool bLowLevellsOn=true, bool bUsePullUpDown=true) -
public-func - void CButton::startMonitoring() -
public-func - void CButton::stopMonitoring() -
public-func - bool CButton::isPressed() -

class CConfigHandler

Configuration handler manager.

Scope - Member - Description

public-func - CConfigHandler::CConfigHandler() -
 public-func - void CConfigHandler::addConfigHandler(String strName, IConfigHandler *pHandler, bool bForceInclude=false) -
 public-func - void CConfigHandler::addConfigHandler(const char *pszName, IConfigHandler *pHandler, bool bForceInclude=false) -
 public-func - IConfigHandler * CConfigHandler::getConfigHandler(String strName, int nIdx=0) -
 public-func - IConfigHandler * CConfigHandler::getConfigHandler(const char *pszName, int nIdx=0) -
 public-func - virtual void CConfigHandler::writeConfigTo(JsonNode &oNode, bool bHideCritical) override - Interface implementation.
 public-func - virtual void CConfigHandler::readConfigFrom(JsonNode &oNode) override -
 public-func - void CConfigHandler::migrateConfig(JsonNode &oCfgDoc, JsonNode &oCfgNode) override -
 public-func - void CConfigHandler::dumpConfigHandler() -

class CEventHandler

Event Handler Manager.

Scope - Member - Description

public-func - CEventHandler::~CEventHandler() -
 public-func - void CEventHandler::registerEventReceiver(IMsgEventReceiver *pEventReceiver, const char *pszReceiverName=nullptr) -
 public-func - int CEventHandler::sendEvent(void *pSender, int nMsgID, const void *pMessage, int nMsgType) - Send the message to the Message Event Receivers To avoid receiving your own message, specify the Sender (this).
 public-func - void CEventHandler::dumpReceiver() -

class CEventLogger

Logger facade that sends formatted log events through CEventHandler.

Scope - Member - Description

public-func - CEventLogger::CEventLogger() - Create a logger without an attached event handler.
 public-func - CEventLogger::CEventLogger(CEventHandler *pEventHandler) - Create a logger that publishes to the given event handler.
 public-func - void CEventLogger::log(const char *pszType, const __FlashStringHelper *pszMessage,...) - Format and publish a flash-string log message.
 public-func - void CEventLogger::log(const char *pszType, const char *pszMessage,...) - Format and publish a C-string log message.
 public-func - void CEventLogger::log(const char *strType, JsonNode *pDoc) - Publish a JSON document as log payload.
 public-static-func - static int CEventLogger::getLogClassNumberFrom(const char cClass) - Convert a log class character into its numeric message class.
 public-static-func - static char CEventLogger::getClassCharFromLogClass(int nClassNo) - Convert a numeric message class back to its log class character.

class CFS

Filesystem facade used by modules to avoid direct LittleFS/SPIFFS use.

Scope - Member - Description

public-func - CFS::CFS() - Create a filesystem helper and initialize the base filesystem if needed.
public-func - bool CFS::fileExists(const char *pszFileName) - Return true if the file exists.
public-func - bool CFS::fileExists(String &strFileName) - String overload for file existence checks.
public-func - void CFS::deleteFile(const char *pszFileName) - Delete a file when it exists.
public-func - void CFS::deleteFile(String &strFileName) - String overload for deleting a file.
public-func - void CFS::deleteAllFilesOnPath(const char *pszPath) - Delete all files below the given path.
public-func - size_t CFS::getFileSize(const char *pszFileName) - Return the file size in bytes, or 0 if it cannot be read.
public-func - size_t CFS::getFileSize(String &strFileName) - String overload for file size lookup.
public-func - FS CFS::getBaseFS() - Return the configured base filesystem object.
public-func - size_t CFS::loadFileToBuffer(const char *strFileName, std::unique_ptr< char[] > &pData) - Load a file into an owned zero-terminated buffer.
public-func - bool CFS::loadFileToString(const char *strFileName, String &strResult) - Load a file into a String.
public-func - bool CFS::loadJsonContentFromFile(const char *strFileName, JsonNode &oDoc) - Parse JSON content from a file into a JsonNode.
public-func - bool CFS::saveJsonContentToFile(const char *strFileName, JsonNode &oDoc) - Serialize and save a JsonNode to a file.
public-func - size_t CFS::getTotalBytes() - Return total bytes of the mounted filesystem.
public-func - size_t CFS::getUsedBytes() - Return used bytes of the mounted filesystem.
public-func - bool CFS::getFileList(JsonNode &oDirDoc, const char *pszPath="/") - Write a directory listing into a JSON node.
public-func - String CFS::getFileList(const char *pszPath="/") - Return a directory listing serialized as text.
public-func - void CFS::writeStatusTo(JsonNode &oStatus, int nLevel=STATUS_LEVEL_INFO) override - Write filesystem capacity/status into a JSON node.

class CInputPinController

InputPinController.h.

(c) 2024 LSC labs InputPinController class Can be initiated with the korrekt pin, level and pullup/down in the constructor or via the setup phase of the project. Respecting the pin logic (logic high = on or off) when asking for the pin status.

Scope - Member - Description

protected-attrb - bool CInputPinController::m_bWithPullUpDown -
protected-attrb - int CInputPinController::m_nMode -
public-func - CInputPinController::CInputPinController() -
public-func - CInputPinController::CInputPinController(int nPin, bool bLowLevelIsOn=true, bool bWithPullUpOrDown=false) -
public-func - virtual void CInputPinController::setup(int SwitchPin, bool bLowLevelIsOn=true, bool bWithPullUpOrDown=false) -
public-func - virtual bool CInputPinController::isPinLogicalOn() -
public-func - virtual bool CInputPinController::isPinLogicalOff() -
public-func - virtual bool CInputPinController::canSendInterrupts() -

class CJsonNode

Scope - Member - Description

public-attrb - String CJsonNode::Name - Optional name/key of this node inside its parent.

public-attrb - std::vector<CJsonNode*> CJsonNode::Elements - Owned child nodes for objects and arrays.

protected-attrb - JsonNode* CJsonNode::m_pParentNode -

protected-attrb - bool CJsonNode::m_bWriteValueWithQuotes -

protected-attrb - ELEMENT_TYPE CJsonNode::m_nObjectType -

protected-attrb - String CJsonNode::m_strSerializationCache -

protected-attrb - String CJsonNode::m_strValue -

protected-func - void CJsonNode::setParentNode(JsonNode *pParentNode) - Store the parent pointer used for tree navigation.

protected-func - const char * CJsonNode::parseValue(const char *pszJsonData, String &strValueData, bool &bHasQuotes) - Parse one scalar JSON token from the input string.

protected-func - const char * CJsonNode::serializeNode(String &strResultString, int nIdentDeep=0) - Recursively serialize this node into the provided string buffer.

protected-func - void CJsonNode::writeIdentPrefixString(String &strResultString, int nIdentDeep=0) - Append indentation spaces used by pretty JSON serialization.

protected-func - void CJsonNode::setNodeValueType(bool bWriteWithQuotes=true) - Convert this node into a scalar value and remember quote handling.

protected-func - bool CJsonNode::getNameFromJsonPath(const char *pszName, String &strName) - Split a dotted JSON path and return the final element name.

public-func - CJsonNode::CJsonNode() - Create an unnamed object node.

public-func - CJsonNode::CJsonNode(const char *pszName, const char *pszValue) - Create a named JSON value node initialized with text data.

public-func - CJsonNode::CJsonNode(const char *pszName, ELEMENT_TYPE eType=ELEMENT_TYPE::OBJECT) - Create a JSON node with an optional name and explicit node type.

public-func - virtual CJsonNode::~CJsonNode() - Delete all child nodes on destruction.

public-func - virtual void CJsonNode::clear() - Delete all child nodes and reset this node to an empty container.

public-func - virtual bool CJsonNode::exists(const char *pszName) - Return true if a direct or dotted-path child exists.

public-func - ELEMENT_TYPE CJsonNode::getType() - Return the stored node type.

public-func - JsonNode * CJsonNode::getParentNode() - Return the parent node, or nullptr for a root node.

public-func - bool CJsonNode::isJsonObject() - Return true when this node is a JSON object.

public-func - bool CJsonNode::isJsonObject(const char *pszName) - Return true when the named child exists and is an object.

public-func - bool CJsonNode::isArray() - Return true when this node is a JSON array.

public-func - bool CJsonNode::isArray(const char *pszName) - Return true when the named child exists and is an array.

public-func - bool CJsonNode::isJsonValue() - Return true when this node is a scalar JSON value.

public-func - bool CJsonNode::isJsonValue(const char *pszName) - Return true when the named child exists and is a scalar value.

public-func - bool CJsonNode::isNumberValue() - Return true if this node contains a simple integer/float literal.

public-func - bool CJsonNode::isNumberValue(const char *pszName) - Return true if a named child contains a numeric literal.

public-func - bool CJsonNode::isBooleanValue() - Return true if this node contains a recognized boolean literal.

public-func - bool CJsonNode::isBooleanValue(const char *pszName) - Return true if a named child contains a recognized boolean literal.

public-func - CJsonNode * CJsonNode::find(const char *pszName) - Find a child by name or dotted

path.

public-func - cJSONNode * cJSONNode::createJsonPathToElement(const char *pszElement) - Ensure that all object nodes before the final path element exist.

public-func - cJSONNode * cJSONNode::createElement(const char *pszName=nullptr) - Create or replace a child value node.

public-func - cJSONNode * cJSONNode::createObject(const char *pszName=nullptr) - Create or replace a child object node.

public-func - cJSONNode * cJSONNode::createArray(const char *pszName=nullptr) - Create or replace a child array node.

public-func - cJSONNode & cJSONNode::operator[](const char *pszName) - Return an existing value element or create it on demand.

public-func - cJSONNode & cJSONNode::operator[](String &strName) - String overload for value element access.

public-func - String & cJSONNode::getValueAsString(String &strDefault) - Return this node's value as String, or the supplied default.

public-func - String & cJSONNode::getValueAsString(const char *pszName, String &strDefault) - Return a named child value as String, or the supplied default.

public-func - const char * cJSONNode::getValue() - Return this node's raw value text.

public-func - const char * cJSONNode::getValue(const char *pszName, const char *pszDefault=nullptr) - Return a named child value or, for value nodes, this node's value.

public-func - const char * cJSONNode::getValueAsCharPointer(const char *pszDefault) - Return this node's value as C string, or the supplied default.

public-func - const char * cJSONNode::getValueAsCharPointer(const char *pszName, const char *pszDefault) - Return a named child value as C string, or the supplied default.

public-func - int cJSONNode::getValueAsInt(const char *pszname, int nDefault=-1) - Parse a named child value as int.

public-func - int cJSONNode::getValueAsInt(int nDefault=-1) - Parse this node's numeric value as int.

public-func - unsigned int cJSONNode::getValueAsUnsignedInt(const char *pszname, unsigned int nDefault=0) - Parse a named child value as unsigned int.

public-func - unsigned int cJSONNode::getValueAsUnsignedInt(unsigned int nDefault=0) - Parse this node's numeric value as unsigned int.

public-func - float cJSONNode::getValueAsFloat(const char *pszname, float fDefault=-999.0) - Parse a named child value as float.

public-func - float cJSONNode::getValueAsFloat(float fDefault=-999.0) - Parse this node's numeric value as float.

public-func - long cJSONNode::getValueAsLong(const char *pszname, long lDefault=0) - Parse a named child value as long.

public-func - long cJSONNode::getValueAsLong(long lDefault=0) - Parse this node's numeric value as long.

public-func - unsigned long cJSONNode::getValueAsUnsignedLong(const char *pszname, unsigned long ulDefault=0) - Parse a named child value as unsigned long.

public-func - unsigned long cJSONNode::getValueAsUnsignedLong(unsigned long ulDefault=0) - Parse this node's numeric value as unsigned long.

public-func - bool cJSONNode::getValueAsBool(const char *pszname, bool bDefault=false) - Parse a named child value as bool.

public-func - bool cJSONNode::getValueAsBool(bool bDefault=false) - Parse this node's boolean value.

public-func - bool cJSONNode::storeValueIf(String &strTarget) - Copy this node's text into the target string when present.

public-func - bool cJSONNode::storeValueIfNot(String &strTarget, const char *pszIfNot) - Copy this node's text unless it matches the excluded marker.

public-func - bool cJSONNode::storeValueIf(const char *pszName, String &strTarget) - Copy a named child text value into the target string when present.

public-func - bool cJSONNode::storeValueIfNot(const char *pszName, String &strTarget, const char

*pszIfNot) - Copy a named child text value unless it matches the excluded marker.

public-func - bool CJsonNode::storeValueIf(int *pnTarget) - Store this value in the target if it is numeric.

public-func - bool CJsonNode::storeValueIf(const char *pszName, int *pnTarget) - Store a named child in the target if it is numeric.

public-func - bool CJsonNode::storeValueIf(unsigned int *pnTarget) - Store this value in the target if it is numeric.

public-func - bool CJsonNode::storeValueIf(const char *pszName, unsigned int *pnTarget) - Store a named child in the target if it is numeric.

public-func - bool CJsonNode::storeValueIf(long *plTarget) - Store this value in the target if it is numeric.

public-func - bool CJsonNode::storeValueIf(const char *pszName, long *plTarget) - Store a named child in the target if it is numeric.

public-func - bool CJsonNode::storeValueIf(bool *pbTarget) - Store this value in the target if it is boolean.

public-func - bool CJsonNode::storeValueIf(const char *pszName, bool *pbTarget) - Store a named child in the target if it is boolean.

public-func - bool CJsonNode::storeValueIf(float *fTarget) - Store this value in the target if it is numeric.

public-func - bool CJsonNode::storeValueIf(const char *pszName, float *pfTarget) - Store a named child in the target if it is numeric.

public-func - CJsonNode * CJsonNode::getObject(const char *pszName, bool bCreateIfNotExist=false) - Return a named object node, optionally creating/convert it.

public-func - CJsonNode * CJsonNode::getArray(const char *pszName, bool bCreateIfNotExist=false) - Return a named array node, optionally creating/convert it.

public-func - CJsonNode * CJsonNode::getElement(const char *pszName, bool bCreateIfNotExist=false) - Return a named scalar value node, optionally creating/convert it.

public-func - const char * CJsonNode::parse(const char *pszJsonData) - Parse JSON text into this node tree.

public-func - void CJsonNode::remove(const char *pszName) - Remove and delete the direct child with the given name.

public-func - CJsonNode * CJsonNode::setValue(const char *pszValue) - Store a quoted string value in this node.

public-func - CJsonNode * CJsonNode::setValue(String &strValue) - Store a quoted String value in this node.

public-func - CJsonNode * CJsonNode::setValue(bool bValue) - Store an unquoted boolean value in this node.

public-func - CJsonNode * CJsonNode::setValue(int nValue) - Store an unquoted signed integer value in this node.

public-func - CJsonNode * CJsonNode::setValue(unsigned int unValue) - Store an unquoted unsigned integer value in this node.

public-func - CJsonNode * CJsonNode::setValue(float fValue) - Store an unquoted floating point value in this node.

public-func - CJsonNode * CJsonNode::setValue(long lValue) - Store an unquoted signed long value in this node.

public-func - CJsonNode * CJsonNode::setValue(unsigned long ulValue) - Store an unquoted unsigned long value in this node.

public-func - CJsonNode * CJsonNode::operator=(const char *pszValue) -

public-func - CJsonNode * CJsonNode::operator=(String &strValue) -

public-func - CJsonNode * CJsonNode::operator=(bool bValue) -

public-func - CJsonNode * CJsonNode::operator=(int nValue) -

public-func - CJsonNode * CJsonNode::operator=(unsigned int unValue) -

public-func - CJsonNode * CJsonNode::operator=(float fValue) -

public-func - CJsonNode * CJsonNode::operator=(long lValue) -

public-func - CJsonNode * CJsonNode::operator=(unsigned long ulValue) -
 public-func - CJsonNode * CJsonNode::setValue(const char *pszName, const char *pszValue) - Set or create a named string value.
 public-func - CJsonNode * CJsonNode::setValue(const char *pszName, String &strValue) - Set or create a named String value.
 public-func - CJsonNode * CJsonNode::setValue(const char *pszName, bool bValue) - Set or create a named boolean value.
 public-func - CJsonNode * CJsonNode::setValue(const char *pszName, int nValue) - Set or create a named signed integer value.
 public-func - CJsonNode * CJsonNode::setValue(const char *pszName, float fValue) - Set or create a named floating point value.
 public-func - CJsonNode * CJsonNode::setValue(const char *pszName, long lValue) - Set or create a named signed long value.
 public-func - CJsonNode * CJsonNode::setValue(const char *pszName, unsigned long ulValue) - Set or create a named unsigned long value.
 public-func - virtual void CJsonNode::dump(const char *pszPrefix="") - Print this node tree to Serial for diagnostics.
 public-func - JsonNode * CJsonNode::createPayloadStructure(const char *pszCommand, const char *pszDataType, const char *pszPayload=nullptr) - Create the common command/data/payload message structure.
 public-func - const char * CJsonNode::getAsJsonText() - Serialize this node tree as compact JSON.
 public-func - const char * CJsonNode::getAsJsonTextPretty() - Serialize this node tree as pretty-printed JSON.

class CLightSwitch

Scope - Member - Description

public-func - CLightSwitch::CLightSwitch() -
 public-func - CLightSwitch::CLightSwitch(int nSwitchPin, bool bLowLevellsOn=true) -
 public-func - void CLightSwitch::blink(unsigned long nOnMillis=1000, unsigned long nOffMillis=1000) -
 -
 public-func - void CLightSwitch::wave(unsigned long ulFadeInMillis=200, unsigned long ulFadeOutMillis=200, unsigned long ulOnTime=1000, unsigned long ulOffTime=1000, int nMaxLevelInPercent=-1) -
 public-func - void CLightSwitch::setBrightness(int nLevelInPercent=100) -
 public-func - int CLightSwitch::getBrightness() -
 public-func - void CLightSwitch::runTests() -

class CLogWriter

Class to handle the logging for a module This class is used to write log entries for a specific module.

Scope - Member - Description

private-func - bool CLogWriter::isTypeToWriteAlways(const char cType) -
 private-func - bool CLogWriter::isLogLevelToWrite(const char cType) -
 public-func - CLogWriter::CLogWriter() -
 public-func - int CLogWriter::setLogLevel(int nLogLevel) - Set the maximum log level written by this writer.
 public-func - int CLogWriter::getLogLevel() - Return the currently configured maximum log level.

public-func - virtual int CLogWriter::receiveEvent(const void *pSender, int nMsgType, const void *pMessage, int nClass) override - Receive log events from the message bus and write matching entries.

public-func - virtual void CLogWriter::writeLogEntry(const char *strType, const char *strMessage) - Write a text log entry to the concrete output.

public-func - virtual void CLogWriter::writeLogEntry(const char *strType, JsonNode *oDoc) - Write a JSON log entry to the concrete output.

class CMDNSController

Scope - Member - Description

public-func - CMDNSController::CMDNSController() -

public-func - CMDNSController::CMDNSController(const char *pszAutoregisterName) -

public-func - CMDNSController::~CMDNSController() -

public-func - bool CMDNSController::begin(const char *pszHostname=NULLptr) -

public-func - void CMDNSController::end() -

public-func - int CMDNSController::receiveEvent(const void *pSender, int nMsg, const void *pMessage, int nClass) override -

class CMQTTController

MQTT message broker module with heartbeat and HA discovery support.

Scope - Member - Description

public-attrb - MQTTConfig CMQTTController::Config - MQTT connection and topic configuration.

public-attrb - MQTTStatus CMQTTController::Status - Current MQTT connection status.

protected-attrb - char* CMQTTController::m_pszHomeAssistantStatusTopic -

public-func - CMQTTController::CMQTTController() - Create the MQTT controller with default config/status.

public-func - CMQTTController::~CMQTTController() - Release owned topic/message buffers.

public-func - void CMQTTController::setup(int nPublishInterval=60) - Configure MQTT callbacks and heartbeat interval.

public-func - void CMQTTController::readConfigFrom(JsonNode &oCfg) override - Read MQTT configuration from a JSON node.

public-func - void CMQTTController::writeConfigTo(JsonNode &oCfg, bool bHideCritical) override - Write MQTT configuration into a JSON node.

public-func - void CMQTTController::writeStatusTo(JsonNode &oCfg, int nLevel) override - Write MQTT connection status into a JSON node.

public-func - int CMQTTController::receiveEvent(const void *pSender, int nMsg, const void *pMessage, int nClass) override - React to application/network lifecycle events.

public-func - void CMQTTController::dispatch() override - Dispatch queued MQTT messages and heartbeat work.

public-func - bool CMQTTController::enableConnection() - Return true if MQTT is enabled and has enough config to connect.

public-func - bool CMQTTController::isDeviceTopic(const char *pszTopic) - Return true if the topic belongs to this device topic prefix.

public-func - bool CMQTTController::isDeviceCommandTopic(const char *pszTopic) - Return true if the topic is a command topic for this device.

public-func - const char * CMQTTController::getDeviceCommandBaseTopicPath() - Return the base topic path used for device commands.

public-func - void CMQTTController::publishDeviceState(JsonNode &oStateData) - Publish the current device state from a JSON node.

public-func - void CMQTTController::publishDeviceState(const char *pszStateData) - Publish the current device state from serialized JSON/text.

public-func - void CMQTTController::publishDeviceTopic(String strTopic, JsonNode &oDataNode, int nQOS=0, bool bRetain=false) - Publish a JSON node below the device topic prefix.

public-func - void CMQTTController::publishDeviceTopic(const char *pszTopic, JsonNode &oDataNode, int nQOS=0, bool bRetain=false) - Publish a JSON node below the device topic prefix.

public-func - void CMQTTController::publishDeviceTopic(const char *pszTopic, const char *pszData, int nQOS=0, bool bRetain=false) - Publish text below the device topic prefix.

public-func - virtual void CMQTTController::publishHeartBeat(bool bForceSend=false) - Publish availability/heartbeat messages when due or forced.

public-func - void CMQTTController::registerHomeAssistantComponent(const char *pszName, IHomeAssistantComponent *pHandler) - Register a component handler for Home Assistant discovery.

protected-func - void CMQTTController::onMqttConnect(bool sessionPresent) - AsyncMqttClient connect callback.

protected-func - void CMQTTController::onMqttDisconnect(AsyncMqttClientDisconnectReason reason) - AsyncMqttClient disconnect callback.

protected-func - void CMQTTController::onMqttMessage(char *topic, char *payload, AsyncMqttClientMessageProperties properties, size_t len, size_t index, size_t total) - AsyncMqttClient message callback.

protected-func - void CMQTTController::publishHomeAssistantDiscovery() - Publish Home Assistant discovery information for registered components.

class CMultiNameUniqueValueTable

Named-value table that allows many names but only unique values.

A value may appear only once in the table. Multiple keys are allowed as long as every key references a different value. set() returns nullptr when a new entry would duplicate an existing value. This is useful for registries where the same object pointer must not be registered twice, even if callers use different names. TValue Type of the stored values.

Scope - Member - Description

public-func - CMultiNameUniqueValueTable< TValue >::CMultiNameUniqueValueTable(bool bCaseSensitiveKey=true, TValue oDefaultValue=0) - Creates a table with unique values and non-unique keys.

class CNamedValueEntry

Single key/value entry used by CNamedValueTable.

The entry owns a copy of the key string. The value is stored directly and is intentionally public so callers can update it after a lookup without another wrapper function. TValue Type of the stored value. The value type must be copyable.

Scope - Member - Description

protected-attr - char* CNamedValueEntry< TValue >::m_pszKey - Owned copy of the key string.

public-attr - TValue CNamedValueEntry< TValue >::value - Stored value.

public-func - CNamedValueEntry< TValue >::CNamedValueEntry(const char *pszKey, TValue value) -

Creates a new entry and copies the key.

public-func - CNamedValueEntry< TValue >::~CNamedValueEntry() - Releases the owned key string.

public-func - const char * CNamedValueEntry< TValue >::getKey() - Gets the stored key.

public-func - bool CNamedValueEntry< TValue >::doesKeyMatch(const char *pszKey, bool bCaseSensitive)
- Checks if a lookup key matches this entry key.

class CNamedValueTable

Lightweight ordered table for named values.

The table stores entries in insertion order and offers lookup by key, lookup by value, and optional uniqueness rules. It is intentionally simpler than std::map: duplicate keys and duplicate values are allowed by default, while derived classes can enable uniqueness constraints. Key comparison can be case-sensitive or case-insensitive. When no matching key exists, get() returns the configured default value. TValue Type of the stored values. The value type must be copyable and comparable with operator== for value lookups and unique-value tables.

Scope - Member - Description

protected-attrb - TValue CNamedValueTable< TValue >::m_defaultValue - Value returned by get() when no matching key exists.

protected-attrb - bool CNamedValueTable< TValue >::m_bCaseSensitive - Controls key comparison mode.

protected-attrb - bool CNamedValueTable< TValue >::m_bOnlyUniqueValues - When true, a value may exist only once in the table.

protected-attrb - bool CNamedValueTable< TValue >::m_bOnlyUniqueKeys - When true, a key may exist only once in the table.

protected-attrb - std::vector<const char *> CNamedValueTable< TValue >::m_tKeyNameCache - Cached list of key pointers in insertion order.

public-attrb - std::vector<CNamedValueEntry<TValue> * > CNamedValueTable< TValue >::Entries - Public entry list for direct iteration.

public-func - CNamedValueTable< TValue >::CNamedValueTable(bool bCaseSensitiveKey=true, TValue defaultValue=0) - Creates an empty named-value table.

public-func - CNamedValueTable< TValue >::~CNamedValueTable() - Deletes all owned entries.

public-func - bool CNamedValueTable< TValue >::hasValueEntry(TValue oValue) - Checks if the table contains at least one entry with a value.

public-func - CNamedValueEntry< TValue > * CNamedValueTable< TValue >::findFirstByValue(TValue oValue) - Finds the first entry with the given value.

public-func - CNamedValueEntry< TValue > * CNamedValueTable< TValue >::findNextByValue(TValue oValue, CNamedValueEntry< TValue > *pLastFound=nullptr) - Finds the next entry with the given value.

public-func - bool CNamedValueTable< TValue >::hasKeyEntry(const char *pszKey) - Checks if the table contains at least one entry with a key.

public-func - CNamedValueEntry< TValue > * CNamedValueTable< TValue >::findFirstByKey(const char *pszKey) - Finds the first entry with the given key.

public-func - CNamedValueEntry< TValue > * CNamedValueTable< TValue >::findNextByKey(const char *pszKey, CNamedValueEntry< TValue > *pLastFound=nullptr) - Finds the next entry with the given key.

public-func - CNamedValueEntry< TValue > * CNamedValueTable< TValue >::findEntryByAddress(CNamedValueEntry< TValue > *pNamedEntry) - Verifies that an entry pointer belongs to this table.

public-func - CNamedValueEntry< TValue > * CNamedValueTable< TValue >::set(const char *pszKey, const TValue value) - Sets or inserts a named value.

public-func - int CNamedValueTable< TValue >::size() - Gets the number of entries in the table.

public-func - std::vector< const char * > CNamedValueTable< TValue >::getKeys() - Gets all known keys in insertion order.

public-func - TValue CNamedValueTable< TValue >::get(const char *pszKey) - Gets the value for a key.

class CNTPHandler

The NTP Handler class as a module.

Scope - Member - Description

protected-attrb - NTPConfig CNTPHandler::Config -
protected-attrb - time_t CNTPHandler::m_oRawTime -
protected-attrb - bool CNTPHandler::m_bIsInitialized -
protected-attrb - unsigned long CNTPHandler::m_uLastUpdate -
protected-attrb - char CNTPHandler::m_szISODateTime[40][40] -
public-func - CNTPHandler::CNTPHandler(const char *pszAutoRegisterName=nullptr) -
public-func - void CNTPHandler::setup() -
public-func - void CNTPHandler::timeUpdatedByService() -
public-func - bool CNTPHandler::hasValidTime() -
public-func - time_t CNTPHandler::getNativeTime() -
public-func - const char * CNTPHandler::getISODateTime() -
public-func - void CNTPHandler::writeConfigTo(JsonNode &oCfgObj, bool bHideCritical) override -
public-func - void CNTPHandler::readConfigFrom(JsonNode &oCfgObj) override -
public-func - void CNTPHandler::writeStatusTo(JsonNode &oStatusObj, int nLevel=STATUS_LEVEL_INFO) override -
public-func - int CNTPHandler::receiveEvent(const void *pSender, int nMsgType, const void *pMessage, int nClass) override -

class COutputPinController

Base class to controll a pin as in or output with helper functions.

Scope - Member - Description

protected-attrb - int COutputPinController::m_nLevelInPercent -
protected-attrb - int COutputPinController::m_nMaxOutputLevel -
protected-attrb - int COutputPinController::m_nState -
public-func - COutputPinController::COutputPinController() - Create an unconfigured output pin controller.
public-func - COutputPinController::COutputPinController(int nSwitchPin, bool bLowLevellsOn=false) - Create and configure an output pin controller.
public-func - virtual void COutputPinController::setup(int SwitchPin, bool bLowLevellsOn=false) - Configure the output pin and active-level logic.
public-func - virtual void COutputPinController::switchOff() - Switch the controlled output off.
public-func - virtual void COutputPinController::switchOn() - Switch the controlled output on.
public-func - virtual bool COutputPinController::toggleSwitch() - Toggle the output and return the new on/off state.
public-func - virtual bool COutputPinController::isOn() - Return true if the output is currently on.
public-func - virtual void COutputPinController::setOutputLevelInPercent(int nLevelInPercent=100) - Set the active PWM/output level in percent.
public-func - virtual int COutputPinController::getOutputLevelInPercent() - Return the configured active PWM/output level in percent.
protected-func - virtual int COutputPinController::getPinLevelValueForPWM(int nLevelInPercent) - Convert a percent value to the hardware PWM range.

class CPinController

PinController.h.

(c) 2024 LSC Labs Base class for Input / Output pins. Defines the pin and if it is active low or active high. Input and Output Controllers should reflect this basic information.

Scope - Member - Description

protected-attr - int CPinController::m_nPin -

protected-attr - int CPinController::m_bLowLevelsOn -

class CRGBLed

Class to handle the RGB LED. Initialize the RGB LED with the pins for the colors and with the information COMMON_ANODE (Active Low) or COMMON_CATHODE (Active High).

The Setup function must be called to initialize the pins

Scope - Member - Description

public-attr - CLightSwitch CRGBLed::RedLED - Red channel light switch.

public-attr - CLightSwitch CRGBLed::GreenLED - Green channel light switch.

public-attr - CLightSwitch CRGBLed::BlueLED - Blue channel light switch.

public-func - CRGBLed::CRGBLed() - Create an unconfigured RGB LED.

public-func - CRGBLed::CRGBLed(int nRedPin, int nGreenPin, int nBluePin, bool bActiveLow=true) - Constructor.

public-func - void CRGBLed::setup(int nRedPin, int nGreenPin, int nBluePin, bool bActiveLow=true) - Configure the RGB LED pins and active-level mode.

public-func - void CRGBLed::switchOn() - Switch all active color channels on.

public-func - void CRGBLed::switchOff() - Switch off the RGB LED.

public-func - void CRGBLed::setColor(RGB_COLOR eColor, bool bSwitchLeds=true) - Set the color of the RGB LED.

public-func - void CRGBLed::setColor(int nColorBits, bool bSwitchLeds=true) - Set the color of the RGB LED - follow the Bitmasks in RGB_COLOR !

public-func - void CRGBLed::blink(RGB_COLOR eColor, unsigned long nOnMillis=500, unsigned long nOffMillis=500) - Blink the given named color.

public-func - void CRGBLed::blink(int nColorBits, unsigned long nOnMillis=500, unsigned long nOffMillis=500) - Blink the given color bitmask.

public-func - void CRGBLed::wave(RGB_COLOR eColor, unsigned long ulFadeInMillis=500, unsigned long ulFadeOutMillis=500, unsigned long ulOnTime=1000, unsigned long ulOffTime=2000, int nMaxLevelInPercent=-1) - Fade the given named color in/out repeatedly.

public-func - void CRGBLed::wave(int nColorBits, unsigned long ulFadeInMillis=500, unsigned long ulFadeOutMillis=500, unsigned long ulOnTime=1000, unsigned long ulOffTime=2000, int nMaxLevelInPercent=-1) - Fade the given color bitmask in/out repeatedly.

public-func - void CRGBLed::setBrightnessInPercent(RGB_COLOR eColor, int nBrightnessPercent) - Set brightness for one named color channel/group.

public-func - void CRGBLed::setBrightnessInPercent(int nColorBits, int nBrightnessPercent) - Set brightness for one color bitmask channel/group.

public-func - void CRGBLed::runTests() - Run the tests for the RGB LED.

public-func - void CRGBLed::showStartupFlashLight(int nDelayBetweenFlashes) - Show a short startup

flash sequence.

private-func - void CRGBLed::runSingleColorTests() - Run single-color diagnostics.

private-func - void CRGBLed::runColorBrightnessTest() - Run color brightness diagnostics.

class CSerialLogWriter

Stream log writer preconfigured for Serial.

Scope - Member - Description

public-func - CSerialLogWriter::CSerialLogWriter() -

class CSimpleDelay

Simple delay class to avoid DOS calls to functions.

Should avoid to call services too often (e.g. web requests). Usage: CDelay oDelay(5000); // 5 seconds
delay if(oDelay.isDone()) { ... ; oDelay.restart(); } As an alternative, you can use
start(nDelayMillis) to set a new delay time. Use reset() to stop the delay.

Scope - Member - Description

public-func - CSimpleDelay::CSimpleDelay() -

public-func - CSimpleDelay::CSimpleDelay(unsigned long nDelayMillis) -

public-func - CSimpleDelay::~~CSimpleDelay() -

public-func - CSimpleDelay * CSimpleDelay::start(unsigned long ulDelayMillis, bool bStartNow=true) -
Starts the delay countdown.

public-func - void CSimpleDelay::stop() -

public-func - void CSimpleDelay::reset() - Resets the delay (stops it).

public-func - bool CSimpleDelay::isActive() - checks if the delay is active (running)

public-func - unsigned long CSimpleDelay::restart() - Restarts the delay with the previous set delay
time.

public-func - unsigned long CSimpleDelay::next() - Advances to the next delay period without
resetting the start time.

public-func - void CSimpleDelay::setExpired() - Set the delay to expired (stop the timer, without
resetting the start time).

public-func - bool CSimpleDelay::isExpired() - checks if the delay is expired (stopped by an
explicit setExpired() - not, if the delay isDone())

public-func - bool CSimpleDelay::isDone() - Checks if the delay time has elapsed, or the user has
stopped the delay with setExpired().

public-func - unsigned long CSimpleDelay::getElapsed() - Get the Elapsed time since start in
milliseconds.

public-func - unsigned long CSimpleDelay::getRemaining() - Get the Remaining time until the delay is
done in milliseconds.

public-func - void CSimpleDelay::printDiag() -

class CStatusHandler

Status handler class.

Works as a status handler and can register further status handler modules.

Scope - Member - Description

public-func - void CStatusHandler::addStatusHandler(String strName, IStatusHandler *pHandler) -
public-func - void CStatusHandler::addStatusHandler(const char *pszName, IStatusHandler *pHandler) -
public-func - IStatusHandler * CStatusHandler::getStatusHandler(String strName) -
public-func - IStatusHandler * CStatusHandler::getStatusHandler(const char *pszName) -
public-func - void CStatusHandler::writeStatusTo(JsonNode &oStatusNode, int nLevel=STATUS_LEVEL_INFO) override - Interface implementation.

class CStreamLogWriter

Log writer that prints log entries to an Arduino Stream.

Scope - Member - Description

public-func - CStreamLogWriter::CStreamLogWriter(Stream *pStream) - Create a log writer for the given stream.
public-func - void CStreamLogWriter::writeLogEntry(const char *strType, const char *strMessage) override - Write a text log entry to the configured stream.
public-func - void CStreamLogWriter::writeLogEntry(const char *strType, JsonNode *pDoc) override - Write a JSON log entry to the configured stream.

class CSysStatus

SysStatus.h.

(c) 2025 LSC Labs Aliases functions to sys information Helps to port information between devices and implements the IStatusHandler interface Not all information are available - still porting... Thanks to the implementation of Jason2866 which gives a lot of basic ideas.
https://github.com/Jason2866/ESP32_Show_Info/blob/main/src/Chip_info.ino#L68

Scope - Member - Description

public-func - CSysStatus::CSysStatus() -
public-func - CSysStatus::~~CSysStatus() -
public-func - const char * CSysStatus::getSdkVersion() -
public-func - const char * CSysStatus::getChipID() -
public-func - uint32_t CSysStatus::getFlashChipId() -
public-func - uint32_t CSysStatus::getFlashChipRealSize() -
public-func - uint32_t CSysStatus::getFlashChipSize() -
public-func - uint32_t CSysStatus::getSketchSize() -
public-func - uint32_t CSysStatus::getFreeSketchSpace() -
public-func - uint32_t CSysStatus::getHeapSize() -
public-func - uint32_t CSysStatus::getFreeHeap() -
public-func - uint32_t CSysStatus::getMaxAllocHeap() -
public-func - uint32_t CSysStatus::getFreePsrAm() -
public-func - uint32_t CSysStatus::getCpuFrequencyMhz() -
public-func - void CSysStatus::writeStatusTo(JsonNode &oStatusObj, int nLevel=STATUS_LEVEL_INFO) override -

class CSysStatusLed

Scope - Member - Description

public-func - CSysStatusLed::CSysStatusLed(int nRedPin, int nGreenPin, int nBluePin, bool bActiveLow=true) -
public-func - virtual int CSysStatusLed::receiveEvent(const void *pSender, int nMsgId, const void *pMessage, int nType) -
public-func - virtual void CSysStatusLed::updateLED() -
public-func - virtual unsigned long CSysStatusLed::getNormalBlinkOnTime() -
public-func - virtual unsigned long CSysStatusLed::getNormalBlinkOffTime() -

class CUniqueNameMultiValueTable

Named-value table that allows unique names and repeated values.

A key may appear only once in the table. Setting an existing key updates the stored value. The same value may be stored under multiple different keys. This is useful for maps where one name must resolve to one current value, but several names may intentionally share that value. TValue Type of the stored values.

Scope - Member - Description

public-func - CUniqueNameMultiValueTable< TValue >::CUniqueNameMultiValueTable(bool bCaseSensitiveKey=true, TValue oDefaultValue=0) - Creates a table with unique keys and non-unique values.

class CVar

Represents one named configuration variable.

Scope - Member - Description

public-func - CVar::~CVar() - Release owned name/key/value buffers.
public-func - CVar::CVar() - Create an empty variable.
public-func - CVar::CVar(const char *pszName) - Create a variable with a name and empty value.
public-func - CVar::CVar(const char *pszName, const char *pszValue) - Create a string variable.
public-func - CVar::CVar(const char *pszName, const int nValue) - Create an integer variable.
public-func - CVar::CVar(const char *pszName, const bool bValue) - Create a boolean variable.
public-func - void CVar::setVarName(const char *pszName) - Set or replace the variable name and derived lookup key.
public-func - const char * CVar::getKeyName(bool bCaseSensitive=false) - Return the lookup key; original case can be requested.
public-func - CVar * CVar::setCriticalVar(bool blsCritical) - Mark whether this variable contains critical/secret data.
public-func - CVar * CVar::setValue(const char *strValue) - Set the variable value from text.
public-func - CVar * CVar::setValue(const int nValue) - Set the variable value from an integer.
public-func - CVar * CVar::setValue(const bool bValue) - Set the variable value from a boolean.
public-func - CVar * CVar::setValue(const unsigned long ulValue) - Set the variable value from an unsigned long.
public-func - bool CVar::isCriticalVar() - Return true if this variable should be masked in public config.

public-func - const char * CVar::getName() - Return the original variable name.
 public-func - const char * CVar::getValue() - Return the value as text.
 public-func - const int CVar::getIntValue() - Return the value converted to int.
 public-func - const unsigned long CVar::getUnsignedLongValue() - Return the value converted to unsigned long.
 public-func - const bool CVar::getBoolValue() - Return the value converted to bool.

class CVarTable

Collection of named variables exposed through IConfigHandler.

Scope - Member - Description

protected-func - bool CVarTable::prepareKeyName(char *szKeyName) -
 protected-func - CVar * CVarTable::getOrCreateVarEntry(const char *strName) -
 public-func - CVarTable::CVarTable(bool bCaseSensitive=false) - Create a variable table with optional case-sensitive lookup.
 public-func - CVarTable::~~CVarTable() - Delete all owned variables.
 public-func - bool CVarTable::hasCriticalVars() - Return true when at least one variable is marked critical.
 public-func - CVar * CVarTable::find(const char *) - Find a variable by name.
 public-func - CVar * CVarTable::find(String) - Find a variable by String name.
 public-func - CVar * CVarTable::set(String strName, String strValue) - Set or create a String variable.
 public-func - CVar * CVarTable::set(String strName, const int nValue) - Set or create an integer variable.
 public-func - CVar * CVarTable::set(String strName, const unsigned long ulValue) - Set or create an unsigned long variable.
 public-func - CVar * CVarTable::set(String strName, bool bValue) - Set or create a boolean variable.
 public-func - CVar * CVarTable::set(const char *pszName, String strValue) - Set or create a variable from C-string name and String value.
 public-func - CVar * CVarTable::set(const char *pszName, const char *pszValue) - Set or create a string variable.
 public-func - CVar * CVarTable::set(const char *pszName, const int nValue) - Set or create an integer variable.
 public-func - CVar * CVarTable::set(const char *pszName, const unsigned long ulValue) - Set or create an unsigned long variable.
 public-func - CVar * CVarTable::set(const char *pszName, bool bValue) - Set or create a boolean variable.
 public-func - const char * CVarTable::getValue(String &strName, const char *pszDefault) - Return a String-named value or the default.
 public-func - const char * CVarTable::getValue(String &strName, String &strDefault) - Return a String-named value or the default String value.
 public-func - int CVarTable::getIntValue(String &strName, int nDefault) - Return a String-named value converted to int.
 public-func - bool CVarTable::getBoolValue(String &strName, bool bDefault) - Return a String-named value converted to bool.
 public-func - const char * CVarTable::getValue(const char *pszName, const char *pszDefault) - Return a value by C-string name or the default.
 public-func - int CVarTable::getIntValue(const char *pszName, int nDefault) - Return a C-string named value converted to int.
 public-func - bool CVarTable::getBoolValue(const char *pszName, bool bDefault) - Return a C-string named value converted to bool.

public-func - virtual void CVarTable::writeConfigTo(JsonNode &oCfgObj, bool bHideCritical) override - Write table values into a JSON config node.

public-func - virtual void CVarTable::readConfigFrom(JsonNode &oCfgObj) override - Read table values from a JSON config node.

public-func - virtual bool CVarTable::hasConfigValues() override - Return true when the table contains at least one variable.

class CWebServer

Async web server module that registers default routes and file access.

Scope - Member - Description

public-attrb - const int CWebServer::Version - Module version exposed for diagnostics.

public-attrb - WebServerConfig CWebServer::Config - Configuration of the webserver.

public-attrb - WebServerStatus CWebServer::Status - Status information about the webserver.

public-func - CWebServer::CWebServer(int nPortNumber=80) - Create a web server on the given TCP port.

public-func - void CWebServer::writeStatusTo(JsonNode &oStatusNode, int nLevel=STATUS_LEVEL_INFO) override - Write web server status into a JSON node.

public-func - void CWebServer::writeConfigTo(JsonNode &oNode, bool bHideCritical) override - Write web server configuration into a JSON node.

public-func - void CWebServer::readConfigFrom(JsonNode &oNode) override - Read web server configuration from a JSON node.

public-func - int CWebServer::receiveEvent(const void *pSender, int nMsg, const void *pMessage, int nClass) override - React to application lifecycle/network events.

public-func - void CWebServer::registerFileAccess() - Register routes used to serve static files from the file system.

public-func - void CWebServer::deliverFile(AsyncWebServerRequest *pRequest) - Deliver a requested file or a fallback response.

public-func - void CWebServer::registerDefaults() - Register built-in routes such as status/config endpoints.

public-func - virtual void CWebServer::setNewAuthHeader(AsyncWebServerRequest *pRequest, AsyncWebServerResponse *pResponse) - Add/update authentication headers on a response.

class CWebSocket

Function pointer to register the routes.

WebSocket endpoint that queues messages and dispatches JSON commands. Incoming multi-frame messages are captured as CWebSocketMessage objects, queued, and later dispatched from the application loop. Selected commands can require authentication before they are processed.

Scope - Member - Description

public-attrb - WebSocketStatus CWebSocket::Status -

public-func - CWebSocket::CWebSocket(const char *pszSocketName, bool bRegisterOnMsgBus=true) - Create a WebSocket endpoint and optionally register on the message bus.

public-func - void CWebSocket::dispatchMessageQueue() - Dispatch all queued WebSocket messages.

public-func - virtual bool CWebSocket::dispatchMessage(CWebSocketMessage *pMessage) - Dispatch one assembled WebSocket message.

public-func - virtual bool CWebSocket::dispatchJsonMessage(JsonNode &oJsonRequest, CWebSocketMessage

*pMessage) - Dispatch one parsed JSON WebSocket request.

public-func - virtual String CWebSocket::setNeedsAuth(const String &strCommands) - Replace the comma-separated list of commands that require auth.

public-func - virtual String CWebSocket::getNeedsAuth() - Return the comma-separated list of commands that require auth.

public-func - virtual bool CWebSocket::needsAuth(String &strCommand) - Return true if the command requires authentication.

public-func - int CWebSocket::receiveEvent(const void *pSender, int nMsgId, const void *pMessage, int nType) - React to application loop/status events.

public-func - virtual void CWebSocket::onWebSocketEvent(AsyncWebSocket *pServer, AsyncWebSocketClient *pClient, AwsEventType eType, void *arg, uint8_t *pData, size_t nLen) - AsyncWebSocket callback used to capture incoming frames/events.

public-func - void ICACHE_FLASH_ATTR CWebSocket::sendAccessDeniedMessage(JsonNode &oDoc, AsyncWebSocketClient *pClient) - Send a JSON access-denied response to a client.

public-func - void ICACHE_FLASH_ATTR CWebSocket::sendJsonDocMessage(JsonNode &oDoc, AsyncWebSocket *pSocket=nullptr, AsyncWebSocketClient *pClient=nullptr) - Serialize and send a JSON document to one client or all clients.

private-func - bool CWebSocket::checkAuth(JsonNode &oRequestDoc, AsyncWebSocketClient *pClient) - Validate request credentials for authenticated commands.

private-func - void CWebSocket::addMessageToQueue(CWebSocketMessage *pMsgObj) - Add an assembled message object to the dispatch queue.

class CWebSocketMessage

Owns a complete WebSocket message assembled from one or more frames.

Scope - Member - Description

public-attr - size_t CWebSocketMessage::MessageSize - Total expected size of the assembled message.

public-attr - char* CWebSocketMessage::pSerializedMessage - Zero-terminated assembled message buffer.

public-attr - int CWebSocketMessage::MessageType - WebSocket message type, for example WS_TEXT or WS_BINARY.

public-attr - AsyncWebSocket* CWebSocketMessage::pSocket - Socket that received the message.

public-attr - AsyncWebSocketClient* CWebSocketMessage::pClient - Client that sent the message.

public-func - CWebSocketMessage::CWebSocketMessage(AsyncWebSocket *pSocket, AsyncWebSocketClient *pClient, size_t nMessageSize, int nType=WS_BINARY) - Construct a new Web Socket Message object.

public-func - CWebSocketMessage::~~CWebSocketMessage() - Release the assembled message buffer.

public-func - CWebSocketMessage * CWebSocketMessage::setMessageData(uint8_t *pData, uint64_t nIndex, size_t nDataLen) - Set the message data (segment).

class CWiFiController

Application module that controls WiFi AP/station mode.

The controller reads/writes its configuration through IConfigHandler, publishes status through IStatusHandler and reacts to WiFi scan requests from the application message bus.

Scope - Member - Description

public-attr - WiFiStatus CWiFiController::Status -

private-func - const char * CWiFiController::getConfigBSSIDAsString() -

private-func - String CWiFiController::getStatusText(int nWiFiStatus) - Convert an Arduino WiFi status code into readable text.

private-func - void CWiFiController::disableWiFi() - Disconnect station/AP mode and publish the corresponding bus events.

private-func - bool CWiFiController::startAccessPoint(const char *pszSSID, IPAddress apip, IPAddress apsubnet, bool bHidden, const char *pszPassword=NULL) - Start access point mode with explicit network settings.

private-func - bool CWiFiController::joinNetwork(const char *pszSSID, const char *pszPassword, byte bSSID[6]) - Join an existing WiFi network in station mode.

private-func - void CWiFiController::onWiFiScanResult(int nNumber) - Build and send the JSON payload for an asynchronous scan result.

private-func - bool CWiFiController::storeIPAddressIf(JsonNode &oCfgData, const char *pszKeyName, IPAddress &pTarget) - Read an IP address from a JSON node when the key exists.

public-static-func - static String CWiFiController::getDefaultSSIDofAP() - Return the default access point SSID generated for this device.

public-func - CWiFiController::CWiFiController() - Create a WiFi controller instance with default configuration/status.

public-func - void CWiFiController::writeConfigTo(JsonNode &oCfgObj, bool bHideCritical) override - Write the current WiFi configuration into a JSON node.

public-func - void CWiFiController::readConfigFrom(JsonNode &oCfgObj) override - Read WiFi configuration values from a JSON node.

public-func - void CWiFiController::writeStatusTo(JsonNode &oStatusObj, int nLevel=STATUS_LEVEL_INFO) override - Write the currently active WiFi state into a JSON status node.

public-func - void CWiFiController::writeStatusToLog() - Serialize the current WiFi status and write it to the application log.

public-func - void CWiFiController::startWiFi(bool bUseConfigData) - Start WiFi using either default AP mode or loaded configuration.

public-func - bool CWiFiController::startAccessPoint(bool bUseConfigData) - Start access point mode.

public-func - bool CWiFiController::restartIfNeeded() - Retry or disable WiFi when a scheduled reconnect is pending.

public-func - void CWiFiController::scanWiFi() - Start an asynchronous WiFi scan.

public-func - int CWiFiController::receiveEvent(const void *pSender, int nMsgId, const void *pMessage, int nType) - Listen to the application message bus and react to WiFi commands.

class IConfigHandler

Interface for configuration handlers.

Scope - Member - Description

public-func - virtual void IConfigHandler::writeConfigTo(JsonNode &oCfgNode, bool bHideCritical)=0 -

public-func - virtual void IConfigHandler::readConfigFrom(JsonNode &oCfgNode)=0 -

public-func - virtual bool IConfigHandler::hasConfigValues() -

public-func - virtual void IConfigHandler::migrateConfig(JsonNode &oCfgDoc, JsonNode &oCfgNode) -

class IHomeAssistantComponent

Implement if needed.

Scope - Member - Description

public-func - virtual void IHomeAssistantComponent::insertComponentDiscovery(const char *pszComponentName, JsonNode &oComponentArea, CMQTTController *pController)=0 - insert your component registration data of HA AutoDiscovery (device).

class IModule

Full Interface for modules.

Configuration HandlingStatus HandlingEvent Handling

class IMsgEventReceiver

Interface for Message Event Receivers.

Scope - Member - Description

public-func - virtual int IMsgEventReceiver::receiveEvent(const void *pSender, int nMsg, const void *pMessage, int nMsgInfo)=0 -

class IPAddress

Scope - Member - Description

public-func - IPAddress::IPAddress() -

public-func - IPAddress::IPAddress(uint8_t n1, uint8_t n2, uint8_t n3, uint8_t n4) -

public-func - void IPAddress::fromString(const char *pszSource) -

public-func - String IPAddress::toString() const -

class IStatusHandler

Interface for a status handler module.

Scope - Member - Description

public-func - virtual void IStatusHandler::writeStatusTo(JsonNode &oStatusNode, int nLevel=STATUS_LEVEL_INFO)=0 -

public-func - virtual bool IStatusHandler::hasStatusValues(int nLevel=STATUS_LEVEL_INFO) - Implement if you don't want to insert your status section, if there is no data in the requested level Default = true, create the section and ask me;

class MQTTMessage

Owns one received MQTT topic/payload pair until dispatch.

Scope - Member - Description

public-attrb - CMQTTController* MQTTMessage::pController - Controller used for topic helper checks; may be nullptr in tests.

public-attrb - char* MQTTMessage::Message - Received message payload from the message broker.

public-attrb - char* MQTTMessage::Topic - Topic that carried the received message.
public-func - MQTTMessage::MQTTMessage(const char *pszTopic, const char *pszMessage, CMQTTController *pController=nullptr) - Copy topic and payload into an owned message object.
public-func - MQTTMessage::~MQTTMessage() - Release copied topic and payload strings.
public-func - bool MQTTMessage::isDeviceTopic() - Return true if this message topic belongs to the device prefix.
public-func - bool MQTTMessage::isDeviceCommandTopic() - Return true if this message topic is a command topic.

class NativeSerial

Scope - Member - Description

public-func - void NativeSerial::printf(const char *pszFormat, Args... args) -
public-func - void NativeSerial::print(const char *pszValue) -
public-func - void NativeSerial::println(const char *pszValue) -

file AccessToken.h

file AESCryptor.h

file Appl.h

Application Main Class.

Peter L. (LSC Labs) 1.0

file ApplModule.h

file base64.h

file Base64Data.h

file BatteryMeasure.h

file Button.h

file ConfigHandler.h

file CWebSocket.md

file DevelopmentHelper.h

file EventHandler.h

file FileSystem.h

file InputPinController.h

file JsonHelper.h

file JsonNode.h

file LightSwitch.h

file Logging.h

file LSCUtils.h

file mainpage.md

file MDNSController.h

MDNS Controller class definition ESP MDNS controller to handle mDNS responder start/stop on WiFi events.

file ModuleInterface.h

Full Interface for a module and an abstract basic module implementation.

LSC-Labs - P.Liebl

file MQTTController.h

file Msgs.h

file NamedValueTable.h

file Network.h

file NTPHandler.h

file OutputPinController.h

file pageBuilder.md

file PinController.h

file ProgVersion.h

file README.md

file RGBLed.h

file Runtime.h

file Security.h

file SimpleDelay.h

file StatusHandler.h

file SysStatus.h

file SysStatusLed.h

file Vars.h

file WebServer.h

file WebSocket.h

file WiFiController.h

WiFi Controller to handle Access Point and Station Mode.

Peter L. (LSC Labs) 1.0 How to use: Instantiate the WiFi Controller "CWiFiController oWiFi" in the global scope Register the controller as a module to the application message bus, and the

configuration / status handling: => CWiFiController oWiFi("wifi"); -or- => CWiFiController oWiFi; => Appl.registerModule("wifi",&oWiFi); -or- => CWiFiController oWiFi; => Appl.MsgBus.registerEventReceiver(&oWiFi); => Appl.addConfigHandler("wifi",&oWiFi); => Appl.addStatusHandler("wifi",&oWiFi); Call startWiFi(true) to start the WiFi with the configuration data, this will enable the WiFi in Station mode, or in Access Point mode. or startWiFi(false) to start a default Access Point

namespace LSC

namespace LSC_WIFI

struct ApplConfig

Runtime configuration of the global CApl object.

Scope - Member - Description

public-attrb - bool ApplConfig::bLogToSerial - true to attach a serial log writer during application init.
public-attrb - bool ApplConfig::bTraceMode - true when trace mode is enabled through configuration.
public-attrb - String ApplConfig::strDeviceName - Device name/hostname used by network-facing modules.
public-attrb - String ApplConfig::strDevicePwd - Device password used by authentication-aware modules.
public-attrb - bool ApplConfig::bDebugFrontend - true to expose frontend debug information.

struct AsyncMqttClientMessageProperties

struct AwsFrameInfo

Scope - Member - Description

public-attrb - bool AwsFrameInfo::final -
public-attrb - uint64_t AwsFrameInfo::index -
public-attrb - uint8_t AwsFrameInfo::num -
public-attrb - uint64_t AwsFrameInfo::len -
public-attrb - uint8_t AwsFrameInfo::opcode -

struct CConfigHandler::HandlerEntry

Scope - Member - Description

public-attrb - const char* CConfigHandler::HandlerEntry::pszName -
public-attrb - IConfigHandler* CConfigHandler::HandlerEntry::pHandler -

struct CStatusHandler::StatusHandlerEntry

Scope - Member - Description

public-attrb - const char* CStatusHandler::StatusHandlerEntry::pszName -
public-attrb - IStatusHandler* CStatusHandler::StatusHandlerEntry::pHandler -

struct LSC_FS

FileSystem helper functions.

This modules helps you to operate with the Filesystem of the ESP32. To avoid programming against LittleFS or SPIFFS, this module provides simple functions to operate with the filesystem, so you can easily switch between this systems. Known issues: Do not store filenames in F("") macros - LittleFS will throw an exception (3). Shared filesystem initialization state.

Scope - Member - Description

public-attrb - bool LSC_FS::isFSInitializedAndOpen - true once the configured filesystem has been mounted/opened.

struct MQTTConfig

Runtime configuration for the MQTT broker connection and topics.

Scope - Member - Description

public-attrb - bool MQTTConfig::isEnabled - true when MQTT should connect/publish.
public-attrb - bool MQTTConfig::useAutoTopic - true to generate the publish topic from device data.
public-attrb - String MQTTConfig::BrokerAddress - Hostname or IP address of the MQTT broker.
public-attrb - int MQTTConfig::BrokerPort - TCP port of the MQTT broker.
public-attrb - String MQTTConfig::ClientID - MQTT client id used by this device.
public-attrb - String MQTTConfig::UserName - MQTT broker username.
public-attrb - String MQTTConfig::UserPassword - MQTT broker password.
public-attrb - String MQTTConfig::PublishTopicPrefix - Base topic used for device state/command topics.
public-attrb - int MQTTConfig::PublishInterval - Heartbeat interval in seconds; 0 disables periodic heartbeat.
public-attrb - bool MQTTConfig::useHA - true to publish Home Assistant MQTT discovery data.
public-attrb - String MQTTConfig::HADiscoveryPrefix - MQTT discovery prefix used by Home Assistant.
public-attrb - String MQTTConfig::HAOnlineMessage - Home Assistant availability message for online.
public-attrb - String MQTTConfig::HAOfflineMessage - Home Assistant availability message for offline.

struct MQTTStatus

Runtime status of the MQTT connection.

Scope - Member - Description

public-attrb - unsigned long MQTTStatus::ConStart - millis() value when connection started.
public-attrb - unsigned long MQTTStatus::ConEnd - millis() value when connection ended.
public-attrb - AsyncMqttClientDisconnectReason MQTTStatus::DisConReason - Last disconnect reason

reported by AsyncMqttClient.

public-attrb - String MQTTStatus::DisConReasonString - Text representation of the last disconnect reason.

struct NTPConfig

NTP Service for ESP Devices Setup when a internet connection (WiFi) becomes available...

: <https://randomnerdtutorials.com/esp32-date-time-ntp-client-server-arduino/> :

<https://forum.arduino.cc/t/getting-time-from-ntp-service-using-nodemcu-1-0-esp-12e/702333/12>

Configuration struct of the NTP Handler

Scope - Member - Description

public-attrb - bool NTPConfig::isEnabled -

public-attrb - String NTPConfig::NTPServer -

public-attrb - String NTPConfig::TZ -

struct WebServerConfig

Runtime configuration of the web server module.

Scope - Member - Description

public-attrb - bool WebServerConfig::AutoRedirectMode - true to redirect unknown/default traffic automatically.

public-attrb - String WebServerConfig::UserName - HTTP basic auth user name; password is read from Appl.

public-func - bool WebServerConfig::authenticate(AsyncWebServerRequest *pRequest, const char *pszReason, bool bLogFailed) - Authenticate a request against configured user and application password.

struct WebServerStatus

Status of the web server module.

Scope - Member - Description

public-attrb - bool WebServerStatus::Started - true after the server has been started.

public-attrb - bool WebServerStatus::AutoRedirectMode - Last active auto-redirect mode.

struct WebSocketStatus

Runtime status of the WebSocket module.

Scope - Member - Description

public-attrb - long WebSocketStatus::uptime - millis() snapshot used as simple uptime/status marker.

struct WiFiConfig

Runtime configuration for access point and station mode.

Scope - Member - Description

public-attrb - bool WiFiConfig::accessPointMode - true for access point mode, false for station mode.

public-attrb - String WiFiConfig::ap_ssid - Access point SSID used when this device creates its own network.

public-attrb - String WiFiConfig::ap_Password - Access point password.

public-attrb - int WiFiConfig::ap_channel - Access point WiFi channel.

public-attrb - bool WiFiConfig::ap_hidden - true to hide the AP SSID from normal scans.

public-attrb - IPAddress WiFiConfig::ap_ipAddress - Static IP address of the access point interface.

public-attrb - IPAddress WiFiConfig::ap_ipSubnetMask - Subnet mask of the access point interface.

public-attrb - String WiFiConfig::wifi_ssid - SSID to join in station mode.

public-attrb - byte WiFiConfig::wifi_bssid[6][6] - Optional fixed BSSID to join; all zero means "do not pin BSSID".

public-attrb - String WiFiConfig::wifi_Password - Password used when joining the configured station network.

public-attrb - bool WiFiConfig::dhcpEnabled - true to use DHCP in station mode.

public-attrb - IPAddress WiFiConfig::ipAddress - Static station IP address used when DHCP is disabled.

public-attrb - IPAddress WiFiConfig::ipSubnetMask - Static station subnet mask used when DHCP is disabled.

public-attrb - IPAddress WiFiConfig::ipGateway - Static station gateway used when DHCP is disabled.

public-attrb - IPAddress WiFiConfig::ipDNS - Static station DNS server used when DHCP is disabled.

public-attrb - bool WiFiConfig::autoFallbackMode - true to fall back to AP mode when station mode cannot connect.

public-attrb - int WiFiConfig::retryTimeoutSeconds - Delay between station reconnect attempts in seconds.

public-attrb - int WiFiConfig::retryCount - Maximum number of reconnect attempts before WiFi is disabled.

struct WiFiStatus

WiFi Status structure.

Scope - Member - Description

public-attrb - unsigned long WiFiStatus::startTimeInMillis - millis() value when WiFi/AP mode was started.

public-attrb - unsigned long WiFiStatus::stopTimeInMillis - millis() value when WiFi was stopped.

public-attrb - unsigned long WiFiStatus::restartTimeInMillis - millis() value for the next scheduled reconnect, or -1 if none.

public-attrb - bool WiFiStatus::isWiFiConnected - true when the active WiFi mode is connected/started.

public-attrb - bool WiFiStatus::isInAccessPointMode - true when the controller currently runs in AP mode.

public-attrb - bool WiFiStatus::isInStationMode - true when the controller currently runs in station mode.

public-attrb - wl_status_t WiFiStatus::wifiStatus - Last Arduino WiFi status, including station connection errors.

public-attrb - int WiFiStatus::nRetryConnectCounter - Number of reconnect attempts already performed.